Волновой коллапс (Wave Function Collapse)

Волновой коллапс (Wave Function Collapse)

Идея: тайлы с правилами

В хабе процедурной генерации мы разобрали несколько подходов: случайное блуждание прорезает тоннели, BSP строит комнаты, клеточные автоматы лепят пещеры, шум Перлина создаёт ландшафт. Но ни один из них не контролирует *эстетику* на уровне отдельных тайлов.

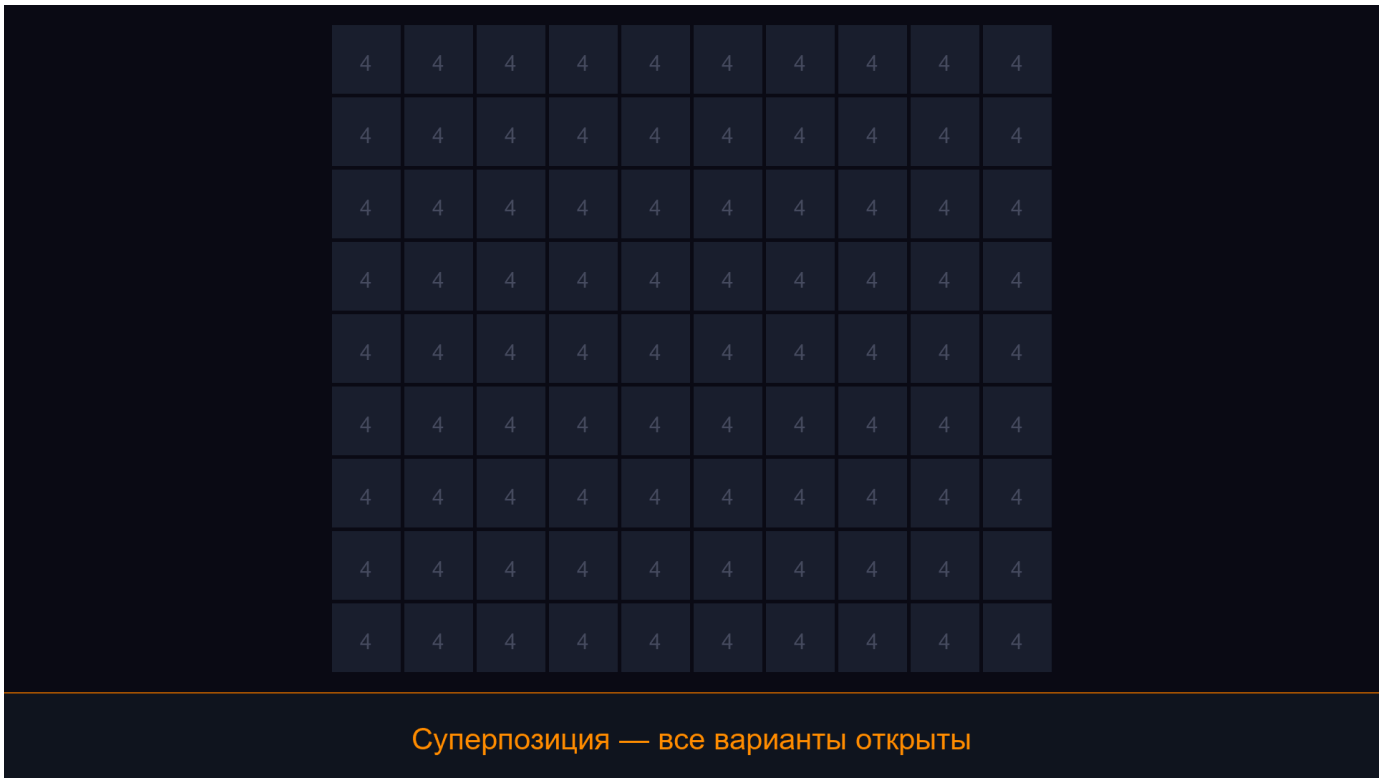
А что, если мы хотим, чтобы вода всегда граничила с песком, песок - с травой, а горы никогда не стояли рядом с водой? Чтобы каждый тайл *логично сочетался* с соседями?

Ну, самый прямолинейный вариант - расставлять тайлы по очереди слева-направо, проверяя совместимость с уже стоящими соседями. Проблема: порядок обхода создаёт "полосатость" - верхние строки влияют на нижние, но не наоборот (имейте в виду, тут мы еще генерируем карту, поэтому все соседи клетки неизвестны, если только мы в будущее не смотрим). А если ни один тайл не подошёл - дыра в карте. А если хочется учитывать соседей с разных сторон? Делать несколько проходов слева-направо, справа-налево, сверху-вниз и т.д.? Но что делать, если соседи из разных проходов будут друг с другом конфликтовать?

Хм. Получается, нужен подход, который учитывает ограничения со всех сторон одновременно.

Именно это делает **Wave Function Collapse** (WFC) - алгоритм, который заполняет сетку тайлами, соблюдая правила соседства. Мы задаём: "вода может стоять рядом с водой и песком, песок - рядом с водой, песком и травой" и т.д. Алгоритм сам находит расстановку, удовлетворяющую всем ограничениям.

Название "волновой коллапс" - метафора из квантовой физики. Каждая ячейка сначала может быть чем угодно (суперпозиция), затем "схлопывается" в одно конкретное значение. Красивая аналогия, но не более - никакой квантовой механики тут нет.



Алгоритм: наблюдай → схлопывай → распространяй

Три шага, повторяемых в цикле:

1. **Наблюдай (Observe).** Найди ячейку с минимальной **энтропией** - наименьшим количеством оставшихся вариантов. Если все ячейки определены - готово.
2. **Схлопни (Collapse).** Выбери случайный тайл из оставшихся вариантов этой ячейки. Теперь она определена.
3. **Распространи (Propagate).** Пройдись по соседям: убери из их вариантов всё, что не совместимо с только что выбранным тайлом. Если у соседа уменьшился набор - проверь *его* соседей. И так далее (BFS/очередь).

Повторяем, пока все ячейки не будут заполнены - или пока не получим **противоречие** (ячейка с нулём вариантов).

Определяем правила

Правила соседства - это словарь: для каждого тайла указываем, кто может стоять рядом. Допустим, у нас четыре типа тайлов - вода, песок, трава, горы:

Python C#

```python
ADJACENCY = {
    'W': {'W', 'S'},        # water <-> sand
    'S': {'W', 'S', 'G'},   # sand <-> water, grass
    'G': {'S', 'G', 'M'},   # grass <-> sand, mountain

```

```
        'M': {'G', 'M'},        # mountain <-> grass
    }
```

Обратите внимание: правила **симметричны**. Если вода может стоять рядом с песком, то и песок может стоять рядом с водой. Это не обязательно - можно сделать асимметричные правила (например, "река течёт только вправо"), но для начала с симметрией будет попроще.

Результат: вода → песок → трава → горы. Естественный градиент - берег, равнина, предгорья. Горы рядом с водой невозможны.

Реализация

Propagate - распространение ограничений

Ключевая часть алгоритма. После схлопывания ячейки нужно обновить соседей. Логика:

1. Кладём схлопнутую ячейку в очередь (BFS).
2. Для текущей ячейки собираем множество **допустимых соседей** - объединение `adj[tile]` для каждого оставшегося тайла.
3. Проходим по четырём соседям. Пересекаем их множества вариантов с `allowed`. Если пересечение пустое - **противоречие** (исключение). Если множество уменьшилось - добавляем соседа в очередь.
4. Повторяем, пока очередь не опустеет.

Т.е. ограничения "расходятся волной" от схлопнутой ячейки - отсюда и название алгоритма.

Энтропия

Почему мы схлопываем ячейку с *наименьшим* числом вариантов? Грубо говоря, она самая "определённая" - ошибка в ней менее вероятна. Если ячейка может быть только водой или песком - выбор из двух. Если может быть чем угодно - выбор из четырёх, и шанс создать проблему выше.

Формально энтропия ячейки: **$H = -\Sigma\, p_i \log(p_i)$**. Для равновероятных вариантов это просто `log(n)`, где `n` - число оставшихся тайлов. На практике для простого WFC `len(options)` работает как эвристика ничуть не хуже.

Та же энтропия из теории информации Шеннона. Формула появляется в деревьях решений (ML), в сжатии данных, в криптографии. WFC - ещё одно место, где она "всплывает".

Python | C#

```python
import math

def entropy(options, weights=None):
    if weights is None:
        return math.log(len(options)) if len(options) > 1 else 0
    total = sum(weights[t] for t in options)
    return -sum((weights[t] / total) * math.log(weights[t] / total)
                for t in options if weights[t] > 0)
```

Если мы хотим, чтобы трава появлялась чаще воды, можно ввести **веса тайлов** и использовать настоящую энтропию для выбора ячейки, а `random.choices` с весами - для схлопывания.

Противоречия и откат

Иногда пропагация загоняет ячейку в тупик: ни один тайл не подходит. Это **противоречие** (contradiction). Ячейка должна быть и рядом с водой (→ только W, S), и рядом с горами (→ только G, M). Пересечение пустое.

Простейшая стратегия: начать заново. Для маленьких сеток (20×15) WFC обычно сходится за 1-2 попытки.

> ⚠ Продвинутый подход - **откат с возвратом** (backtracking): запоминаем состояние перед каждым схлопыванием и при противоречии откатываемся к предыдущему шагу. Это гарантирует нахождение решения (если оно существует), но существенно сложнее в реализации.

Влияние правил

Количество и строгость правил кардинально меняют результат:

- **Мало правил** (все тайлы совместимы со всеми): хаотичная мешанина, почти как наивный рандом. Горы рядом с водой, трава посреди океана.
- **Строгие правила** (наш пример W→S→G→M): чёткий градиент, красивые переходы, но предсказуемый результат.
- **Много тайлов + сложные правила**: дороги, реки, мосты, леса. Сложнее настроить, но результат впечатляет.

Т.е. тот же алгоритм, разные правила - разные миры. Как и с Boids: один код, разные параметры.

Плюсы и минусы

Плюсы:

- **Контроль дизайнера.** Правила задаются вручную - вы решаете, что с чем сочетается.

- **Локальная согласованность.** Каждый тайл гарантированно подходит к соседям.
- **Расширяемость.** Добавить новый тайл = добавить строку в словарь правил.
- **Красивый результат.** Тайловые карты выглядят "дизайнерскими", а не случайными.

Минусы:

- **Ручное описание правил.** Для 20 типов тайлов - 20 строк правил. Для 100 - уже утомительно.
- **Рестарты.** Противоречия неизбежны, особенно при строгих правилах.
- **Нет глобальных ограничений.** WFC не гарантирует, что карта проходима, что есть река от края до края, что горы формируют хребет. Только локальная согласованность.
- **Скорость.** Пропагация на больших сетках может быть медленной. Для реального времени нужна оптимизация.

Для проверки проходимости после WFC используйте flood fill из хаба процедурной генерации.

WFC - это, по сути, **задача удовлетворения ограничений** (CSP, Constraint Satisfaction Problem):

- **Переменные** = ячейки сетки.
- **Домены** = множество допустимых тайлов для каждой ячейки.
- **Ограничения** = правила соседства.

Шаг "propagate" - это **arc consistency** (AC-3): убираем из доменов соседей значения, несовместимые с текущим. Та же техника, что решает Судоку, расписания, раскраску графов.

Если вы встретите CSP в курсе ИИ или дискретной математики - вспомните WFC. Те же принципы, тот же алгоритм, только вместо чисел в клетках - тайлы на карте.

Что дальше?

Что	Где подробнее
Другие подходы к генерации (BSP, CA, Perlin, Random Walk)	Процедурная генерация
Клеточные автоматы - тоже "локальные правила → глобальный результат"	Клеточные автоматы
BSP - структурированные подземелья без правил соседства	BSP-генерация
Плавный ландшафт и биомы через шум	Шум Перлина
Формальные грамматики для растений и фракталов	L-системы
Seed, валидация, flood fill	Процедурная генерация

Что	Где подробнее
Управление случайностью, веса, shuffle bag	Случайность и вероятность
Поиск пути по сгенерированной карте	Поиск путей

Powered by Wiki.js